

TRINITY COLLEGE · SEPTEMBER 21, 2026

AI Integration

Inside the harness, and your first API call

Ken Kousen · CPSC 415
Monday 1:30–4:10 · MC-225

Where we are

Last week

Agent built the app
You changed one thing
You checked it
You explained one piece

This week

Your program calls the model
You see what the harness sends
You write intent before code
You verify with the bill



Two or three trainers from the tag, on the screen, first.

Model, harness, application

Part	Week 1	Week 2
Model	Selected through OpenRouter	Same, or a local one
Harness	Claude Code	Claude Code
Application	HTML page, no AI calls	Your program calls the model

Today the application makes the call, so you get to read the call.

The loop, with the tools named

Read → propose → act → observe → decide

In the Week 1 demo

The tool that ran

Looked at `lyrics.txt`

Read a file

Created `index.html`

Write a file

Started a local server

Run a shell command

Opened the page

Drive a browser

Permissions are a boundary, not a verdict

Mode	Reads	Edits	Commands
Default	Yes	Asks	Asks
Auto	Yes	Asks for risky ones	Asks for risky ones
Plan	Yes	No	No
Bypass	Yes	Yes	Yes

Approving an action says it may run.
It does not say the result is right.

Plan mode

Read `intent/chat-client.md`.

Tell me your plan: which file, what it contains.

Wait for my approval before writing.

Plan mode makes that request structural: the harness will not edit until you say so.

Week 1 asked for a plan in the prompt. Plan mode enforces it.

Context is everything the model sees

Supplied by the harness

System instructions

CLAUDE.md

Tool definitions

Files it read

Supplied by you

Your prompts

Your corrections

The conversation so far



Every turn sends all of it again.

The window fills

Turn 1 is cheap.

Turn 40 carries turns 1 through 39.

Window	Where you see it
Published size	The model's page on OpenRouter
Current use	<code>/context</code> inside the session
Compaction	The harness summarizes to make room

A model not in the harness's catalog is assumed to have a 200K window; the `[1m]` suffix from Week 1 tells it otherwise.

When to start fresh

- The task changed. New session, new goal.
- The session is arguing with itself. Summarize what worked, start over with the summary.
- Compaction happened twice. What survived may not be what you needed.
- You want a second opinion. A fresh session, or a different model, has no sunk cost.



Write the handoff note before you close the session.

CLAUDE.md

Chat client

- Python 3, standard library only. No pip installs.
- Read configuration from environment variables; never hardcode a key.
- Before editing, state the plan and wait.
- After editing, show how to run it.



Project memory, read before your first prompt.

Parameters

7B fits a laptop.

27B wants a lot of memory.

Hundreds of billions live in the cloud.

Parameters are the trained weights between the layers, not the nodes.
Inference uses them; training is the other course.

Dense and mixture of experts

Kind	Loads	Runs per token
Dense	All parameters	All of them
Mixture of experts	All parameters	A chosen subset

A listing such as "2.4T total, 95B active" is a mixture-of-experts model: the whole thing is stored, a fraction is used for each token.

Open weights, not open source

Published

The weights
A license
Often the architecture

Usually not published

Training data
Training process
Evaluation details



You can run it. You cannot rebuild it.

Long context has a price

- Providers quote **per million tokens**, input and output separately.
- Some charge a **higher rate past a context threshold**.
- **Cached input** can be cheaper when the same prefix repeats.

Read the pricing page for the model you are using, today.

Numbers change monthly. The slide has none on purpose.

What a chat request contains

```
{  
  "model": "vendor/model-name",  
  "messages": [  
    {"role": "system", "content": "Answer briefly for a college student."},  
    {"role": "user", "content": "What is a context window?"}  
  ],  
  "max_tokens": 600,  
  "temperature": 0.2  
}
```



Sent as JSON to `/chat/completions` with your key in a header.

What comes back

```
{  
  "model": "vendor/model-name",  
  "choices": [  
    {"message": {"role": "assistant", "content": "A context window is ..."},  
     "finish_reason": "stop"}  
  ],  
  "usage": {"prompt_tokens": 33, "completion_tokens": 41}  
}
```



The `usage` block is what you pay for.

The `content` is what you check.

Parameters you can set

Parameter	What it does
<code>max_tokens</code>	Caps the output, including any hidden reasoning
<code>temperature</code>	Lower favors the most likely next token; higher spreads the choice
<code>stream</code>	Sends the answer as it is generated
<code>system</code> message	Sets the frame every answer is generated inside

Same names across most providers. Defaults differ. Read the docs for yours.

Same request, different destinations

Destination	Base URL	Key
OpenRouter	<code>https://openrouter.ai/api/v1</code>	Your OpenRouter key
Local model	<code>http://localhost:PORT/v1</code>	Any string
A vendor directly	The vendor's URL	That vendor's key

One program. Change two environment variables.

Running a model on your own machine

Tool	What it is
Ollama	Command line and background service; pull, run, serve
LM Studio	Desktop app: model browser, chat window, local server
llama.cpp	The engine under many of these; loads model files directly

All three serve the request shape you just saw. None is required for this course.

Where a small model falls short

Slower. Shorter memory. Less reliable on hard questions.

Free after the download, and private.



Show it. Do not assert it.

For some components it is good enough. That is a design decision you will document.

Intent before spec before plan

Goal · Who it is for · Constraints

Not in scope · Success looks like · Open questions



The agent drafts it. You approve it. Then code.

The discovery interview

I want to build a small chat client.

Before writing anything, interview me.

Ask one question at a time until you can fill in every section of intent/TEMPLATE.md.

Then write your draft to intent/chat-client.md and stop.

"I don't know" is an answer. It goes under Open questions.

Correct the draft

- It assumed a package you did not ask for. Strike it; say "standard library only."
- It promised streaming. Move it to **Not in scope**.
- It wrote a success criterion it cannot check. Replace it with one you can.

Sign it. Approved by: your name, date.

Interview, intent, build

Template repository → interview → approve →
plan → run

Python or Java. Standard library only. Key from the environment.

The full lab is in [weeks / 02 / lab.md](#).

Verify the call

Evidence	Where
The model that answered	OpenRouter Activity, not the program's print statement
Tokens in and out	Activity page beside your program's usage line
The request shape	Ask the agent to show the JSON it sends
A parameter's effect	<code>max_tokens</code> at 20; changed system prompt

The response's description of itself is not evidence.

Swap the model

Change `CHAT_MODEL`. Same question.

Write down both answers and both costs.

Then, if you have one running: change `CHAT_BASE_URL` to localhost.

One question, two observations. Not a benchmark.

Save evidence of the work

<code>intent/chat-client.md</code>	Drafted by the agent, corrected and signed by you
<code>chat.py</code> or <code>Chat.java</code>	Reads the environment, sends one question
<code>CHECKS.md</code>	Six checks, expected and observed
<code>README.md</code>	Corrections, one code explanation, the comparison



Commit. Tag `week02-submitted`. Push both.

Before next Monday

September 28 · 1:30 PM

- Submit your repository URL and `week02-submitted` tag.
- Upload your **AI Fluency for Students** certificate.

Next week: from intent to `spec.md`, structured output, a five-case eval, and **teams are formed**. Team Project 1 and the portfolio intent are assigned.

Thinking about an Elting Center sandbox project instead? Tell Ken this week.

Before you leave

What did the interview get wrong?

What did your program send?

What did the bill say?



Three answers, then go.

Sources and next steps

- Ken Kousen, *Claude Code: Up and Running*: Chapters 3–5 and Chapter 11 on context and the harness. Draft readings via Moodle.
- [Artifact-chain template](#): the intent template used today.
- [The AI-Native SDLC Playbook](#): the Plan stage.
- [OpenRouter quickstart](#) · [Ollama OpenAI compatibility](#) · [LM Studio](#) · [llama.cpp](#)
- [Claude Code training](#) · [Codex training: plan-mode warm-up](#)
- Reference clients rehearsed September 15 against a local Ollama model; hosted rehearsal before class.

Prepared September 15, 2026. Recheck permission-mode names, local-server ports, and pricing pages before class.